

Supporting Information — Empiria: A Composable, Browser-Based Environment for Simulation-Based Statistics Education

May 2026

This Supporting Information documents Empiria’s numerical validation, reproducibility procedures, computational footprint, module library, and bundled lessons. It supplements the main article; everything reported here is openly available in the repository and can be regenerated by a reviewer in a few minutes. Section references (§) point to the main article.

Appendix A — Numerical validation against R

Empiria’s pedagogical claim rests on its numbers being *exactly right*, not “good enough for a demo.” Every statistical routine is checked against an authoritative reference implementation to the stated tolerance — to machine precision unless a Monte-Carlo tolerance is noted (§ Numerical correctness and reproducibility).

A.1 Reproducing the validation (a reviewer, in three commands)

```
npm install
npm test           # 41 automated checks against R, all green
npm run verify     # the same numerics, printed beside their R values
npm run benchmark  # engine throughput (Appendix B)
```

To confirm the *reference values themselves* — not merely that the code agrees with our own expectations — run the independent R script and compare its printout to the literals asserted in `test/numerics.test.ts`:

```
Rscript verification/reference_values.R
```

A.2 Closed-form quantities

Quantity	Algorithm / recipe	Reference (R)	Test
Regularized incomplete beta $I_x(a, b)$	Lentz modified continued fraction (Press et al. 2007, §6.4)	<code>pbeta()</code>	numerics: incomplete beta
Student- t two-tailed p	$I_{df/(df+t^2)}(df/2, \frac{1}{2})$, exact	<code>2*pt(- t , df)</code>	numerics: student-t p
Student- t critical value	bisection on the exact tail	<code>qt(1-alpha/2, df)</code>	numerics: t critical

Quantity	Algorithm / recipe	Reference (R)	Test
Incomplete gamma / χ^2 tail	series + continued-fraction switch (§6.2)	<code>pchisq(..., lower.tail=F)</code>	numerics: chi-square tail
Normal CDF	Abramowitz & Stegun 7.1.26 <code>erf</code>	<code>pnorm()</code>	numerics: normal cdf
Normal quantile	Acklam rational approx. (rel. err < 1.15e-9)	<code>qnorm()</code>	numerics: normal quantile
One-sample t -test	$t = (\bar{x} - \mu_0)/(s/\sqrt{n})$, exact p	<code>t.test(x, mu)</code>	numerics: one-sample
Welch two-sample t	unequal-variance t , Welch–Satterthwaite df	<code>t.test(a, b)</code>	numerics: Welch
Simple OLS (β , α , R^2)	closed-form least squares	<code>lm(y ~ x)</code>	numerics: OLS
Contingency χ^2 + Cramér’s V	Pearson χ^2 (no Yates correction)	<code>chisq.test(m, correct=F)</code>	numerics: contingency
Autocorrelation $\rho(k)$	n-divisor sample ACF	<code>acf(x)\$acf</code>	numerics: autocorrelation
Quantiles	linear interpolation	<code>quantile()</code> type 7	numerics: summary
Skewness / excess kurtosis	moment estimators	<code>e1071::skewness/kurtosis</code>	numerics: summary
BCa bootstrap interval	bias-correction z_0 + jackknife accel. (Efron 1987)	<code>boot::boot.ci("bca")</code> (to MC error)	engine: coverage
RNG	MT19937 (Matsumoto & Nishimura 1998)	canonical test vector	numerics: MT19937

A.3 Behavioural / statistical-property checks

These assert that the *system* behaves correctly, not just that individual formulas evaluate:

- Determinism. The same master seed produces a byte-identical signal stream; a different seed produces a different one.
- MT19937 test vector. Seed 5489 $\rightarrow$ first output 3499211612, the documented reference output of the canonical generator.
- Law of Large Numbers. A growing sample’s mean $\rightarrow \mu$ and SE $\rightarrow 0$ ($\approx 1/\sqrt{n}$).
- Node matches library. A node’s reported statistic equals a direct library call on its own buffer to < 1e-9.
- CI coverage. Over ~500 repeated experiments, the nominal-95% t -interval covers the true mean 90–99% of the time.
- Type-I error. With two identical populations, the rejection rate sits near α (≈ 2 –9% for $\alpha = 0.05$).
- Every bundled lesson and tour step applies to a fresh graph and ticks without error.

Appendix B — Computational footprint

Empiria runs entirely in the browser: there is no server, and after the initial download — a single static bundle of roughly 0.4 MB (about 130 KB gzipped) — it runs offline. The numerical workload is light and

scales linearly in the number of nodes. The benchmark below is reproducible with `npm run benchmark`.

Measured on an Apple M4 (single-threaded; Node v25), warmed up. The interactive on-screen clock is capped at a few thousand ticks/second, so for realistic patches the render loop — not the arithmetic — is the limiting factor, leaving ample headroom on low-power classroom hardware.

Patch	Engine throughput	Per tick
Typical (2 nodes)	$\approx 2,600,000$ ticks/s	$0.38 \mu\text{s}$
Busy canvas (16 nodes)	$\approx 330,000$ ticks/s	$3.0 \mu\text{s}$
Busy canvas (64 nodes)	$\approx 59,000$ ticks/s	$17 \mu\text{s}$
Very large (256 nodes)	$\approx 6,700$ ticks/s	$149 \mu\text{s}$
Extreme (1024 nodes)	≈ 530 ticks/s (~80 MB heap)	1.9 ms

Appendix C — Reproducibility and cross-machine determinism

- All randomness flows from a seeded MT19937 (`src/engine/rng.ts`); `Math.random()` is never used.
- A patch is plain JSON (parameters, wiring, master seed). Loading it on any machine reconstructs identical initial conditions, computed in IEEE-754 double precision with no platform-specific code paths. Patches save as files or as shareable URLs that encode the whole simulation.
- Independent audit of a worked result. Open any patch, set the master seed, run it, and use a node’s CSV export. Recompute the statistic in R/Python (`t.test`, `lm`, `chisq.test`, ...): it reproduces Empiria’s on-screen value to six decimals, and the same seed on a different machine yields a byte-identical export.

Scope of validation (limitations). The BCa bootstrap is validated by its *coverage behaviour* and by exact tests of its components (z_0 via the normal quantile; the jackknife acceleration), not by a value-for-value match to `boot::boot.ci`, which would require reproducing R’s resampling RNG stream. Confidence intervals use the t distribution (exact at small n); intervals for proportions use the normal (Wald) form.

Appendix D — Use of AI coding assistance

Empiria’s implementation was written with the help of AI coding assistants (large language models). The validation in Appendix A exists precisely so that correctness does not depend on that fact: every routine is checked against an independent authoritative reference (R, the `boot` package, the canonical MT19937 test vector), and the reference values are regenerated by `verification/reference_values.R` rather than asserted from the code’s own output. A check that compares to R passes only if the number is right, regardless of how the code was authored. The statistical methods, the choice of exact recipes over normal-theory approximations, the pedagogical claims, and the interpretation of all results were specified, reviewed, and are vouched for by the author; the AI tools accelerated implementation and did not make statistical decisions. The author takes full responsibility for the software and the manuscript.

Appendix E — Module library

Modules (“nodes”) are grouped by role. Each emits its result as a control-voltage signal that can be routed into any compatible input, and (with the formula view on, § Pedagogical rationale) displays the quantity it computes in standard notation.

Role	Modules (what each computes)
Sources	Sample (i.i.d. draws from Normal / Uniform / Exponential, with running mean and SD); Data (CSV) (streams an imported dataset); Seed (global master seed); Note (annotation)
Estimation & summary	Frame (windowed/growing mean, SD, standard error); Summary (mean, median, SD, IQR, skew, kurtosis); Box (median, IQR; box-and-whisker); Means (batch means, for the CLT)
Inference	Test (one-sample and Welch two-sample t : t , p , decision, Cohen's d); Boot (BCa bootstrap: estimate, 95% interval, SE); Coverage (repeated-experiment CI coverage); Power (rejection rate; Type-I error and power); Tab (contingency χ^2 , p , Cramér's V)
Diagnostics & EDA	ECDF (empirical CDF vs Normal); QQ (normal quantile–quantile plot); Lag (autocorrelation $\rho(k)$ with Bartlett bands); Scope (time-series trace)
Transformation & combination	Transform ($y = a \cdot f(x) + b$); Noise ($y = x + N(0, \sigma^2)$); Mix (mixture, sum, or mean of two streams); Code (continuous $\rightarrow$ ordinal / Likert)
Readout	Gauge (a labelled real-world instrument readout)

Appendix F — Bundled lessons and guided tour

Each lesson loads as a self-contained worksheet — a patch plus an explanatory Note — so it opens ready to run and to modify.

1. Law of Large Numbers
2. Bootstrap sampling distribution
3. t -test with a small sample
4. Regression — what no relationship looks like
5. Survey responses (Likert coding)
6. Bring your own data (CSV)
7. Mixture of two distributions
8. Comparing two groups (Welch t)
9. CLT from a skewed parent
10. Build a real relationship (Transform + Noise)
11. What “95% confidence” means
12. The CLT, directly (batch means)
13. Statistical power & sample size
14. False positives when nothing is going on
15. Is it Normal? (QQ plot)
16. Explore a dataset (EDA)

A four-step guided tour (Law of Large Numbers $\rightarrow$ bootstrap $\rightarrow$ t -test $\rightarrow$ confidence-interval coverage) introduces the canvas with previous/next navigation.

References

The references cited above appear in full in the main article: Abramowitz & Stegun (1964); Efron (1987); Matsumoto & Nishimura (1998); Press, Teukolsky, Vetterling & Flannery (2007).